

Algorithms and Data Structures Computation II

Instructions

Answer each of the following multiple-choice questions by selecting the best answer (A–E).

1. (1 pts) Which of the following has the biggest growth rate?

- | | |
|----------------------------|---------------------------|
| A. $O(n \log n)$ | C. $O(n^2 \sqrt{\log n})$ |
| B. $O(\sqrt{n}(\log n)^2)$ | D. $O(n \log(n^2))$ |

2. (1 pts)

You are given the following Python code snippet:

```
1 n = 100
2 for i in range(n):
3     j = 1
4     while j < n:
5         print(i, j)
6         j *= 3
```

How many pairs (i, j) will this code print?

- | | |
|--------------------|---------------------|
| A. $O((\log n)^3)$ | D. $O(n \log n)$ |
| B. $O(\log(n/3))$ | E. $O(\sqrt[3]{n})$ |
| C. $O(n)$ | |

3. (1 pt) Consider the following Python code, which implements a recursive function:

```

1 def algarisms(n):
2     if n < 10:
3         return n
4     else:
5         return n % 10 + algarisms(n // 10)
6
7 print(algarisms(345))

```

Recall that `n % k` returns the remainder of dividing n by k , and `n // k` returns the integer quotient (floor division).

What is the output of this code?

- A. 10^3
 - B. 10
 - C. 12
 - D. 34500
 - E. 345
4. (1 pts) You are running a **binary insertion sort** on an array of integers. After 4 iterations of the algorithm, which of the following arrays could represent its current state?
- A. [2, 3, 5, 8, 10, 4, 1, 9]
 - B. [2, 3, 8, 5, 10, 1, 4, 9]
 - C. [2, 5, 3, 8, 10, 1, 4, 9]
 - D. [8, 5, 3, 2, 10, 1, 4, 9]
 - E. [1, 2, 3, 6, 5, 4, 7, 8]
5. (1 pts) What is the main difference between Binary Insertion Sort (BIS) and regular Insertion Sort?
- A. BIS uses a heap to speed up insertions to an overall $O(n \log n)$ time complexity
 - B. BIS uses binary search to find the insertion point, which improves the search step to $O(n \log n)$ time complexity.
 - C. BIS always sorts in descending order
 - D. BIS has an overall complexity $O(n \log n)$ instead of $O(n^2)$ due to the binary search.
6. (2 pts) Let us revisit the Towers of Hanoi (or virus problem from the practical sessions thereof). Suppose you need to transport the 5 disks from peg A to peg C . After that, you need to transport the largest disk (i.e., the bottom one) back to peg A from peg C . What is the minimum number of moves required to complete the task?

Remark that the minimum number of moves to move n disks from one peg to another (obeying the constraints) can be characterized recursively as:

$$T(n) = 2T(n - 1) + 1 \quad \text{with} \quad T(1) = 1.$$

A. 43

D. 46

B. 44

E. 47

C. 45

10. (1 pts) As discussed in class, 'Self' defines a variable to be at the object scope. Consider the following Python class definition:

```

1 class Counter:
2     def __init__(self, start):
3         self.value = start
4
5     def increment(self):
6         value = self.value + 1
7
8     def get(self):
9         return self.value

```

What is the output of the following code?

```

1 c = Counter(10)
2 c.increment()
3 c.increment()
4 print(c.get())

```

- A. 10
B. 11
C. 12
D. 13
E. Error: value is not defined
11. (1 pts) Consider the following Python class definition that overrides the special operator method `__eq__()`:

```

1 class Vector:
2     def __init__(self, x, y, z):
3         self.x = x
4         self.y = y
5         self.z = z
6
7     def __eq__(self, other):
8         return self.x == other.x and self.z == other.z

```

Now consider the following code:

```

1 v1 = Vector(1, 2, 3)
2 v2 = Vector(1, 99, 3)
3 print(v1 == v2)

```

What will this code print?

- A. True
- B. False
- C. None
- D. Error: == not supported for Vector
- E. Error: AttributeError on z

12. (1 pts) Consider the following Python class definition:

```
1 class Vector:
2     def __init__(self, x, y, z):
3         self.x = x
4         self.y = y
5         self.z = z
```

Now suppose we create an object:

```
1 v = Vector(1, 2, 3)
```

What does the variable `v` actually store?

- A. The values of the attributes `x`, `y`, and `z`.
- B. A list containing `[1, 2, 3]`.
- C. The address in memory where the
- D. A copy of the class definition with attributes 1, 2 and 3.
- E. A tuple containing `(1, 2, 3)`.

13. (2 pts) Consider the following Python class hierarchy:

```
1 class Logger:
2     def log(self):
3         print("Logging from Logger")
4
5 class FileLogger(Logger):
6     def log(self):
7         print("Writing to file...")
8         super().log()
9
10 class SecureFileLogger(FileLogger):
11     def log(self):
12         print("Encrypting log...")
13         super().log()
```

Now suppose we run:

```
1 logger = SecureFileLogger()
2 logger.log()
```

What is the output of this code?

- | | |
|--|--|
| <p>A. Encrypting log...
Writing to file...
Logging from Logger</p> <p>B. Encrypting log...
Logging from Logger
Writing to file...</p> <p>C. Writing to file...</p> | <p>Encrypting log...
Logging from Logger</p> <p>D. Writing to file...
Logging from Logger
Encrypting log...</p> <p>E. Error: <code>super().log()</code> cannot be chained more than once</p> |
|--|--|

14. (1 pts) What is a key advantage of using linked lists (over standard arrays)?

- | | |
|--|--|
| <p>A. $O(1)$ access time to any element (as opposed to $O(n)$ in arrays)</p> <p>B. Binary Insertion Sort over Linked Lists is $O(n \log n)$ due to efficient insertion (whereas in arrays, insertion is $O(n)$).</p> <p>C. $O(1)$ insertion and deletion at known</p> | <p>positions (as opposed to $O(n)$ in arrays)</p> <p>D. All elements are stored contiguously in memory (as opposed to customized in arrays)</p> <p>E. It is capable of storing more data than arrays.</p> |
|--|--|

15. (1 pts) Consider the following implementation of a singly linked list node:

```

1 class Node:
2     def __init__(self, data):
3         self.data = data
4         self.next = None

```

Now consider the following sequence of operations:

```

1 a = Node("A")
2 b = Node("B")
3 c = Node("C")
4 d = Node("D")
5
6 a.next = b
7 b.next = c
8
9 d.next = c
10 b.next = d
11
12 a.next = d
13
14 def print_list(node):
15     if node:
16         print(node.data)
17         print_list(node.next)
18

```

19 `print_list(a)`

What is the output of the code?

- | | |
|------|------|
| A. A | D |
| D | |
| C | D. A |
| | C |
| B. A | D |
| B | |
| D | E. A |
| C | D |
| C. A | B |

16. A max-heap is a complete binary tree whose values stored at the nodes obey an order: $A[i] \geq A[j]$ for all indexes j that represent a child of i . Preserving a max-heap order is quite important for sorting efficiently.

MAX-HEAPIFY. Suppose that the left and right subtrees rooted at the children of i are already max-heaps. Then, the **MAX-HEAPIFY**(i) operation restores the max-heap property in the subtree rooted at i by performing a maximum of $h(i)$ swaps where $h(i)$ is the height of the subtree rooted at node i —defined as the number of edges on the longest path from i to a leaf in its subtree.

Prove by induction that the **MAX-HEAPIFY**(i) operation performs at most $h(i)$ swaps to restore the max-heap property, assuming the left and right subtrees of i are already max-heaps.

- a. **Base case.** Consider the base case where the children of i are leaves (or $h(i) = 1$). (1 pts)
- b. **Induction step.** Assume that the claim holds true for $h(i) = m$ (i.e., for any subtree of height m) and prove that this should imply it is true for $h(i) = m + 1$ (1pts)